What Should an Inspection Drone Fleet Learn? Tool Selection, Answer Fusion and Federated Reinforcement Learning under Time Budgets

Mahdi Khashan

[a][Affiliation to be added],

Abstract

An inspection drone can ask several perception tools about a frame: a fast onboard classifier, onboard vision-language models (VLMs), or a large VLM in the cloud over a weak link. Agentic designs let a VLM decide which tool to call and read the results, and a fleet could train this behaviour with federated reinforcement learning. We test this design and its simpler alternatives on 13,729 power-line component items from ElecVQA-Bench (InsPLAD), with measured tool times, LTE uplink traces and a cost of 10 per missed defect and 1 per false alarm. Three findings stand out. First, a VLM agent does not benefit from its tools: with a 201M-parameter vision-language decision model (laya-vision) reading zoom, classifier and cloud outputs, PPO training of its scorer or of its 8.65M-parameter decision head leaves the report worse with tools than without. Second, most of the value lies in fusing the tools' own answers: a logistic fusion with a label-free threshold from EM prior correction lowers the test risk from 177.0 for the best single tool to 107.1 per 1,000 items. Third, a PPO policy that chooses which tool to call next under a time budget reaches the risk of calling every tool (105.9 ± 1.5 against 105.4, 3 seeds) in 30% less time, and FedAvg over five drones matches central training with 28 KB per update and lower seed variance at tight budgets. The policy does not, however, beat a one-line uncertainty cascade (106.3), also not when drones differ in components and network quality, where FedAvg with local fine-tuning is the most stable learned method. An oracle that knows the best tool subset per item reaches 65.0; this gap lies in the state the policy sees, not in the learning algorithm. A routing controller federated across operator regions under one-sided feedback, and federated fine-tuning of the perception model (0.4–41 GB per run), complete the picture of what a fleet can learn per transmitted byte.

Keywords: drone fleets, federated learning, reinforcement learning, tool selection, vision-language model, edge AI, infrastructure inspection

1. Introduction

Infrastructure inspection is moving from drones that record video for later analysis to fleets that interpret what they see on board [1]. For each frame, a drone can ask perception tools of very different cost: a fast onboard classifier, a slower onboard VLM, or a larger VLM in the cloud reached over a link that is often weak or down. Farahani et al. run a fine-tuned SmolVLM-256M on a Jetson in 1.4–6.4 s per segment [2] and profile whether such a model should run on board, at the edge or split [3]; both name an adaptive runtime orchestrator as future work.

Two designs for such an orchestrator are popular. In the agentic design, a VLM is the agent: it looks at the frame, decides to zoom or to ask another model, reads the result and writes the report. Reinforcement learning can train this behaviour, and a fleet can share it through federated learning. In the routing design, a small policy only decides which tool to call next; the report is computed from the tools' answers. Both designs promise budget-aware behaviour, and both are cheap to federate compared with the perception models themselves.

This paper tests these designs on cached tool outputs for power-line inspection, with measured tool times, LTE uplink traces and a cost that weights a missed defect ten times a false alarm. We ask what a fleet should learn: the agent, the policy that selects tools, the controller that routes frames, or the perception model. Our answer is partly negative, and we report it as such.

We contribute:

- evidence that a vision-language decision model used as an agent does not profit from its tools, across six PPO configurations that train its scorer or its 8.65M-parameter head, centrally and with FedAvg (Section 4);
- a fusion of tool answers with a label-free threshold from EM prior correction [12], which lowers the test risk from 177.0 to 107.1 per 1,000 items and is the largest single gain we measured (Section 5);
- a tool-selection environment with time budgets and an uplink model, in which PPO [11] and FedAvg-PPO

Email address: mahdikhashan1@gmail.com (Mahdi Khashan)

learn budget-aware policies that match calling every tool in 30% less time, but do not beat a tuned uncertainty cascade, also for drones that differ in components and network (Section 6);

- a federated routing controller under one-sided feedback and a comparison with federated fine-tuning of the perception model, which together give the cost of each option in transmitted bytes (Sections 7 and 8).

2. Related work

Edge VLMs for inspection drones. EVLM [2] runs a LoRA-tuned SmolVLM-256M on a Jetson and cuts the uplink of a 4 s segment from 485 kB to 25 kB; [3] profiles on-board, edge and split deployment, and INAR-VL [7] routes requests between edge and cloud with a bandwidth guard. All use fixed rules. Laya [5] and laya-vision [6] answer typed questions in one forward pass with calibrated probabilities instead of generating text; we use laya-vision both as a tool and as an agent.

Routing, cascades and partial feedback. Federate the Router [17] learns language-model routers with FedAvg from offline evaluations; FedHLM [18] federates escalation thresholds. BaRP [9] trains a router from bandit feedback. Zhang et al. [8] study hierarchical inference when the probability of reaching the ground-truth oracle depends on the policy. Agarwal et al. [10] evaluate federated contextual bandits. Our routing policy differs in that it can call several tools in sequence under a time budget and the report fuses all answers.

Reinforcement and federated learning. PPO [11] with a clipped objective is our learner. FedAvg [14] averages client models weighted by data size; non-IID data degrades it [15], FedProx [16] adds a proximal term, and Flower [13] provides the simulation used for the agentic experiments. Label shift between training and deployment is corrected with the EM procedure of Saerens et al. [12].

3. Data, tools and cost

We use the tower14 split of ElecVQA-Bench [19], built on InsPLAD power-line component crops [20]: 18 training towers (25,499 items), 10 validation towers (6,672) and 14 test towers (7,057). Each item asks whether a component has a defect, or which condition it has. The validation towers hold 19.6% defects and the test towers 15.1%, so any threshold tuned on validation is biased for test. Every tool was run once on every item and the experiments replay these cached answers:

- ResNet: a ResNet-50 [22] fine-tuned on the training towers;
- laya: laya-vision [6] (201M parameters) fine-tuned on 4,807 training images (item accuracy 0.948; 0.939 in 4-bit at 189 MB);
- SmolVLM: SmolVLM-256M [4] with LoRA [23] as in EVLM;
- Qwen: Qwen3-VL-8B in the cloud, zero-shot, which needs a JPEG upload (median 33 kB, mean 40 kB).

Onboard times are RTX 2080 Ti or GV100 times multiplied by 9, our estimate of the gap to a Jetson Orin NX: ResNet 0.07 s, laya 0.70 s and SmolVLM 0.73 s per item on average, and Qwen computes in 0.13 s. A Qwen call costs its compute time, a 50 ms round trip and the upload over 4G/LTE traces [21] scaled to a weak regime (10% of the trace bandwidth, six 20 s outages per hour); during an outage Qwen is unavailable. The risk per 1,000 items counts 10 per missed defect, 1 per false alarm and 0.5 per abstention, so the Bayes threshold on a calibrated defect probability is 1/11.

Table 1: A VLM agent gets worse with tools. Report cost per 1,000 test images (lower is better; defect risk plus component and condition errors), base laya-vision in 4-bit as agent and report writer, 400 test images, seed 0, 1 s budget, round 10. Steps: mean actions per image.

Method	Report cost	Defect recall	Steps
Zero-shot	1,668	0.57	1.0
PPO trains the shared option scorer			
Report only, no tools	1,505	0.77	1.0
Central, with tools	1,715	0.50	4.9
FedAvg, with tools	1,628	0.55	6.7
PPO trains the decision head (8.65M parameters)			
Report only, no tools	1,438	0.66	1.0
Central, with tools	1,570	0.48	1.2
FedAvg, with tools	1,725	0.41	3.8

4. A VLM as the agent

We first tested the agentic design. Laya-vision answers a typed question by scoring its options in one forward pass, so it can serve as a policy: we add the actions (zoom into one of five overlapping crops, ask ResNet, ask Qwen, report) as options and read the action from its scores. Tool results return as text in the next query. PPO trains the policy on cached frozen backbone outputs of 800 training images and is evaluated on 400 test images at budgets of 1, 3 and 6 s; FedAvg runs in Flower over five drones. The report cost counts the defect risk plus errors on the component and condition questions.

Table 1 shows the outcome. In every configuration, the report written with tools is worse than the report written without them. Separate scorers for the action and the report questions did not change this (1,645–1,765 with tools against 1,592 without). Training the decision head gave the only budget-dependent behaviour: the central policy learned to stop after 1.2 steps at 1 s and 2.0 steps at 3 s, but its reports remained worse than without tools. FedAvg learned about five times more slowly and cost 8.65 MB per drone and round in int8. A pilot with the fine-tuned laya-vision as reader gave the same picture: PPO learned to

Table 2: Fusing the tools' answers is the largest gain. Defect risk per 1,000 test items (7,057 items, 1,068 defects) with 95% bootstrap intervals. Single tools use their own argmax answer.

Method	Risk	95% CI
ResNet alone	177.0	148–206
SmolVLM alone	184.9	157–211
laya alone	261.0	225–294
Qwen alone	625.8	575–679
Fusion, threshold tuned on validation	115.8	100–133
Fusion, Bayes threshold 1/11	113.8	99–130
Fusion, Bayes after EM correction	107.1	91–124
Fusion, EM correction per region	117.5	100–137
Fusion, best threshold on test (labels)	105.4	89–124
Best single tool per item (labels)	44.8	30–60

call no tool, because each tool turned more correct answers into wrong ones than the reverse.

Two further checks explain part of this. When every tool answers laya's three questions directly, fusing their answers improves component accuracy from 0.872 to 0.905 (oracle 0.925), but SmolVLM fails outside its training prompt (component accuracy 0.14) and zero-shot Qwen calls almost every image defective (false-alarm rate 0.95). And test-time zoom does not help even the classifier: over five overlapping crops, the best zoom rule raises the image-level test risk from 110.0 to 126.9; the defects ResNet misses are small source crops (median 62k against 110k pixels), so zoom adds no detail. The tools are useful, but not through a reader that has to interpret their output.

5. Fusing the tools' answers

The alternative is to combine the tools' own defect probabilities. For each item we form the logit of $P(\text{defect})$ of every called tool, a flag per tool, the task and the component type, and fit a logistic regression on the validation towers; a missing tool enters as logit 0 with flag 0, so the same model scores every subset of tools. Regularization and features are chosen by five-fold cross-validation grouped by tower.

The threshold is the difficult part. A threshold tuned on validation is too low for the test towers, which hold fewer defects. Without labels, we estimate the test defect share $\hat{\pi}$ with the EM procedure of Saerens et al. [12], rescale every probability from the validation share π_v to $\hat{\pi}$,

$$p' = \frac{p\,\hat{\pi}/\pi_v}{p\,\hat{\pi}/\pi_v + (1-p)(1-\hat{\pi})/(1-\pi_v)},$$

and apply the Bayes threshold 1/11. EM is only a mean over posteriors, so a fleet can compute it federatedly.

Fusion lowers the risk from 177.0 for the best single tool to 113.8, and the EM-corrected threshold to 107.1, within 2 points of the best threshold chosen on test labels (Table 2). EM estimates a test defect share of 0.154 against the true 0.151. Estimating it per region is worse (117.5), because small regions give unstable estimates. The remaining gap to the per-item oracle (44.8) consists mostly of false alarms: in an analysis with a less regularized fusion, 719 of 725 false alarms had at least one tool answering correctly, and 31 defects were flagged by no tool at all.

6. Learning which tool to call

6.1. Environment

Fusion requires calling every tool, which costs time and uplink. We therefore let a policy choose which tools to call. An episode is one item. ResNet always runs first. The policy then chooses among calling laya, calling SmolVLM, calling Qwen, or writing the report; a tool is available only if it has not been called and fits in the remaining budget. The report is the fused, EM-corrected decision from the tools called so far. The reward is

$$r = -\,\text{risk} - \lambda\,\frac{t}{B},$$

paid when the report is written, where t is the time used and B the budget, drawn from $\{0.25, 0.75, 1.5, 3\}$ s. The state holds the current fused logit and its distance to the threshold, the ResNet logit, the logits of the tools called so far, the remaining budget, the current bandwidth and link state, the cost of each tool, and image statistics (size, blur, contrast, brightness) with task and component.

The policy is a multilayer perceptron with two hidden layers of 64 units (6,916 parameters, 28 KB in fp32), trained with PPO (clip 0.2, entropy 0.01, Monte-Carlo returns minus a learned value) for 30 rounds of 2,048 episodes on the validation towers. Its fused probabilities come from a fusion fitted out of fold, so the policy never sees probabilities from a model that saw the item's labels. Tests use the test towers with the fusion fitted on all validation towers. FedAvg PPO splits the validation towers into five drones of balanced size; each runs one PPO update per round and the server averages the actors weighted by data size. We compare with ResNet alone, calling every tool that fits, and an uncertainty cascade that calls the next tool (laya, then SmolVLM, then Qwen) while $|p - 1/11| \le 0.1$ and fewer than two tools have been called. An oracle chooses the best subset per item from the labels.

6.2. Results

PPO learns a sensible, budget-aware policy (Fig. 1, Table 3). It calls laya first, more often as the budget grows (26–60% of items at 0.75 s, 65–100% at 3 s), and at 3 s reaches the risk of calling every tool in 30% less time. ResNet alone, calibrated by the same fusion, has a risk of 146.9. FedAvg matches central training and is more stable: at 0.75 s it reaches 123.4 ± 0.8 against 129.5 ± 8.7. The time weight matters: with $\lambda = 0.2$ the policy correctly learns never to call a tool, since the time penalty exceeds the average gain per item. At 0.25 s only Qwen fits, and calling it slightly increases the risk (150.6).

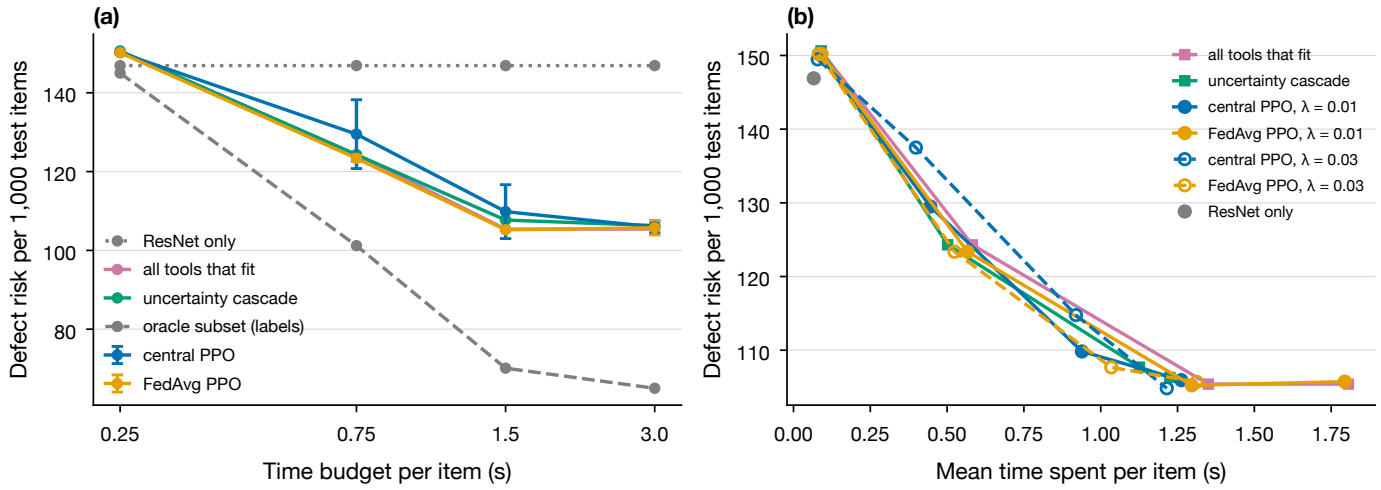


Figure 1: PPO learns budget-aware tool selection and matches calling every tool in less time, but not more accurately than a cascade. (a) Defect risk per 1,000 test items at each time budget, weak LTE regime; PPO lines are means over 3 seeds after 30 rounds, whiskers one standard deviation, time weight $\lambda = 0.01$. (b) The same methods as risk against the mean time spent per item; filled markers $\lambda = 0.01$, open markers $\lambda = 0.03$. The oracle is omitted from (b); it needs 0.11 s on average.

Table 3: Tool selection at the 3 s budget, test towers, weak network. PPO: mean ± standard deviation over 3 seeds. Qwen: share of items sent to the cloud.

Method	Risk	Time (s)	Qwen
ResNet only	146.9	0.07	0
All tools that fit	105.4	1.81	0.92
Uncertainty cascade	106.3	1.23	0
Central PPO, $\lambda = 0.01$	105.9 ± 1.5	1.26	0.58
Central PPO, $\lambda = 0.03$	104.8 ± 3.4	1.21	0.61
FedAvg PPO, $\lambda = 0.01$	105.7 ± 1.8	1.79	0.92
FedAvg PPO, $\lambda = 0.03$	105.7 ± 1.8	1.31	0.53
Oracle subset (labels)	65.0	0.11	0.02

Table 4: Drones with different component groups and network regimes: fleet risk per 1,000 test items (7,057 items), mean ± standard deviation over 3 seeds.

Method	0.75 s	1.5 s	3 s
Static cascade	126.5	109.3	106.3
Cascade tuned, one rule	126.5	108.3	107.3
Cascade tuned per drone	127.4	120.2	117.1
Central PPO	138.1 ± 8.3	116.1 ± 6.1	107.7 ± 1.4
Local PPO	131.7 ± 6.9	110.0 ± 2.0	107.1 ± 2.3
FedAvg PPO	126.5 ± 0.0	113.0 ± 5.4	110.8 ± 4.6
FedAvg + fine-tune	126.6 ± 0.1	112.0 ± 2.6	107.9 ± 0.9
Oracle (labels)	105.1	73.6	66.8

The policy does not beat the cascade. In Fig. 1b, central PPO, FedAvg PPO and the cascade lie on the same curve of risk against time. Calling a further tool helps when the fused probability is close to the threshold, and the one-line cascade already encodes this. The oracle shows that much more is possible in principle (65.0), but it exploits per-item knowledge of which tool will correct the answer, which none of the state features predict.

6.3. Drones that differ

A fixed rule should suffer when drones differ, which is the case federated learning is meant for. We assign each of five drones one component group (glass insulators, polymer insulators, hardware, yokes, rods and grips; 795–1,912 validation items each) and its own network regime (poor, good, weak, poor, good), and repeat the experiment with three further methods: local PPO on each drone alone, FedAvg PPO followed by 10 rounds of local fine-tuning, and cascades whose tool order, margin and maximum number of tools are tuned on validation, either one rule for the fleet or one per drone. A second split assigns tower groups instead of components. The fusion and the EM estimate are shared by the fleet; per-drone EM failed, estimating 12% defects for the hardware drone, which has 0.8%.

Heterogeneity does not change the conclusion (Fig. 2, Table 4). FedAvg with local fine-tuning is the most stable learned method and at 3 s matches the cascade (107.9 ± 0.9 against 106.3) in 1.05 s instead of 1.23 s per item, but no method lowers the risk. Tuning the cascade per drone is clearly worse (117.1), because a drone's validation and test towers differ: the glass drone sees 35% defects in validation and 20% in test. The rule tuned for the whole fleet keeps the order laya, SmolVLM, Qwen at every budget, which suggests that the best order of tools hardly depends on the component. The tower split gives the same ranking.

7. A federated routing controller under one-sided feedback

The tool-selection experiments give the policy the true risk of every episode. In deployment, a drone sees the label of a frame only if an operator reviews it. We studied this in a mission replay with one decision per frame: after the classifier, a 29 KB multilayer perceptron (7,171 parameters) chooses to accept, ask the onboard SmolVLM,

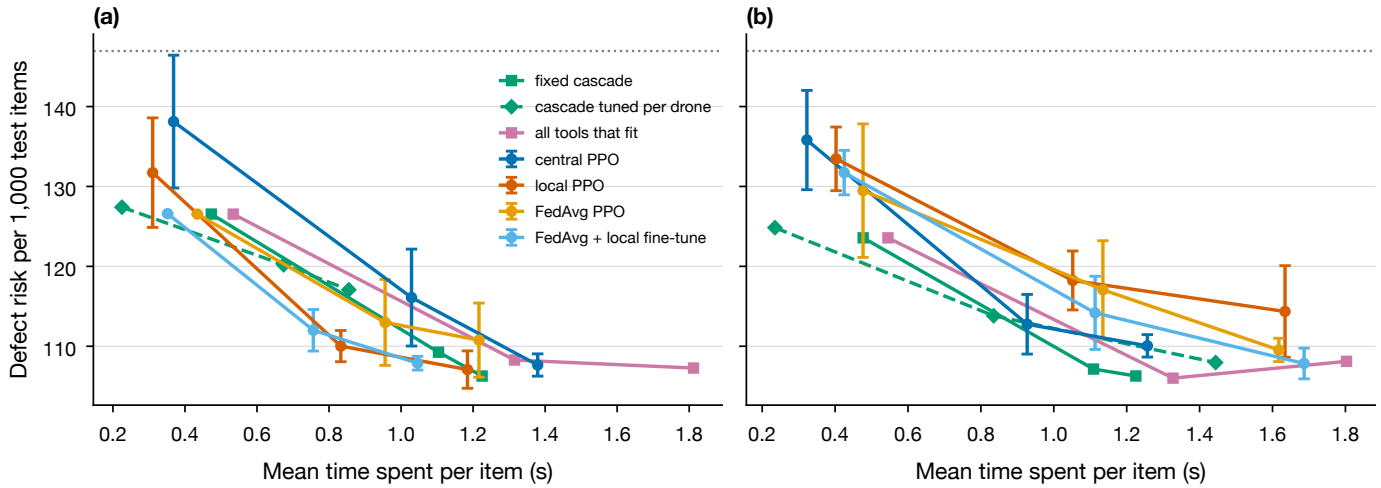


Figure 2: When drones differ in components and network, learned policies still do not beat the cascade; FedAvg with local fine-tuning is the most stable. Fleet defect risk per 1,000 test items against mean time per item, at budgets of 0.75, 1.5 and 3 s (left to right along each line), $\lambda = 0.01$. (a) Each drone inspects one component group. (b) Each drone covers one tower group. Learned methods: means over 3 seeds, whiskers one standard deviation. Dotted line: ResNet only.

Table 5: Routing controllers on the test towers, weak network, mean ± standard deviation over 3 training seeds (5 trace seeds each). Uploads: frames sent by audits and escalations while learning.

Controller	Risk	Labels	Uploads
Classifier only	177.0	–	–
Static cascade	155.7	–	–
Oracle (hindsight)	62.9	–	–
Full feedback, offline			
Central	114.6 ± 3.1	25,499	–
FedAvg	115.4 ± 1.6	25,499	–
Regions alone	157.9 ± 20.4	4,579–6,449	–
One-sided feedback, online, federated			
Cloud as truth, $\lambda = 5\%$	175.7 ± 2.1	2,443	141 MB
True labels, $\lambda = 0$	149.6 ± 5.4	386	30 MB
True labels, IPW, $\lambda = 5\%$	142.4 ± 3.6	1,727	76 MB
Multi-fidelity, $\lambda = 2\%$	156.8 ± 5.3	1,052	58 MB
One-sided feedback, online, regions alone			
True labels, IPW, $\lambda = 5\%$	157.3 ± 5.3	2,047	101 MB

or send the frame to the cloud, subject to a 4 s deadline, an onboard queue and the weak uplink. Clients are five operator regions that keep their images and labels, built from the 18 training towers (4,579–6,449 frames each, two with a single tower).

With labels for every frame, FedAvg reaches 115.4 ± 1.6, equal to central training (114.6) and 26% below the static cascade, for 6.7–7.9 MB of total uplink (Fig. 3a, Table 5). Regions alone are no better than the cascade (157.9 ± 20.4); the two single-tower regions never learn to escalate. Unlike the tool-selection policy, this controller sees the classifier's full probability vector and the component, and chooses between tools whose errors differ strongly, which leaves room to learn.

Deployment feedback is one-sided: accepting a frame reveals nothing, the cloud answer is a noisy label (accuracy 0.79), and an operator labels a frame only when it is sent and reviewed. An audit sends accepted frames to the operator with probability λ. Learning from cloud answers makes the controller worse than the cascade (175.7), because the cloud is wrong on 1,322 test items where the classifier is right. Without audits, operator labels are too few (386 across the fleet). Only federated learning from operator labels with a 5% audit beats the cascade (142.4 ± 3.6 with inverse-propensity weights); regions alone with the same audit do not (157.3). The audit uploads (76 MB) exceed the controller updates (28.7 MB).

8. Federating perception instead

The last option is to federate the perception model and keep a fixed rule. Zero-shot, laya-vision fails on this data (condition macro F1 0.180); fine-tuned centrally, it reaches macro F1 0.703 and defect AUROC 0.965 at 74 ms per image. We federated it in 42 configurations: 3 to 18 drones, the decision head, LoRA of rank 8, the last 4 or all 20 language layers, or the full model, in fp16 or int8, from the published checkpoint or from a server model fine-tuned on 13 server-held towers, with few labels per drone and with drones that see different components. Each configuration has one seed.

One full update is 115 MB in int8, about 4,000 times the 28 KB update of the tool-selection policy, and a training run sends 0.4–41 GB against about 4 MB for 30 rounds of policy training on five drones (Fig. 4, Table 6). From the server model every update lifts defect AUROC from 0.903 to 0.940–0.962, but the small updates double the calibration error (0.06 to 0.14–0.15), because each fleet drone's images hold 34–67% defects while the test towers hold 10%. As in the fusion, the defect share differs between where a model is trained and where it is used, and

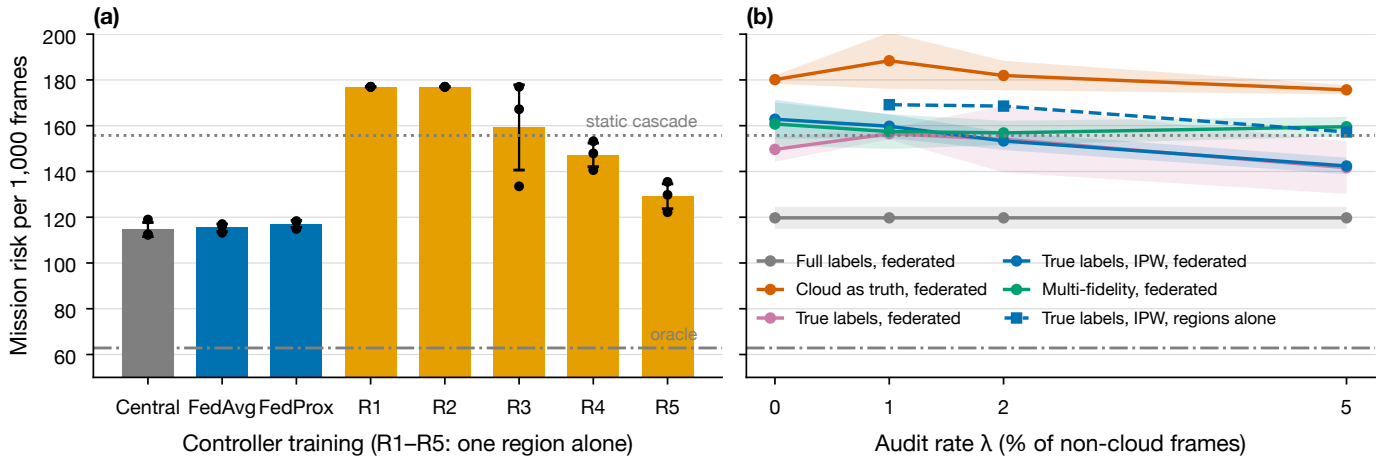


Figure 3: A federated 29 KB routing controller matches central training; under one-sided feedback only audited operator labels, pooled across regions, beat the cascade. Mission risk per 1,000 test frames, weak network. (a) Full feedback (offline): bars are means over 3 training seeds, dots the seeds, whiskers one standard deviation. R1–R5 train alone. (b) Online learning over 10 rounds against the audit rate λ; bands are one standard deviation over 3 seeds. Dotted line: static cascade; dash-dotted line: oracle.

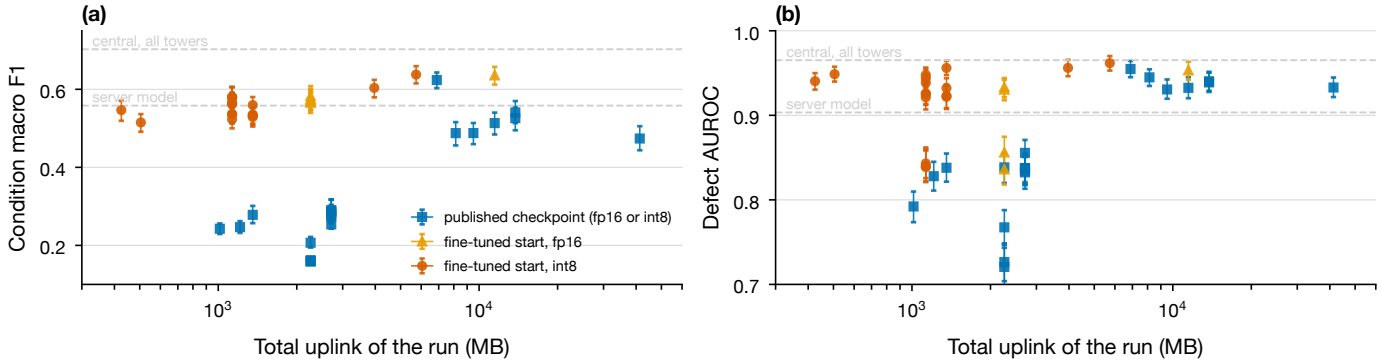


Figure 4: Federating the perception model costs gigabytes, and the start matters more than the uplink. Test condition macro F1 (a) and defect AUROC (b) of laya-vision for 42 federated configurations against the total uplink of the run, by start. Whiskers are 95% bootstrap intervals; dashed lines mark central training and the server model. One seed per run.

Table 6: Federated laya-vision from the server model (5 fleet drones, 10 rounds, int8 updates, one seed). F1: condition macro F1. ECE of the defect probability, raw and after a Platt fit on 1,500 validation images.

Update	Uplink	F1	AUROC	ECE	Platt
Server model	0	0.558	0.903	0.063	0.027
Head	425 MB	0.547	0.940	0.149	0.027
LoRA r8	505 MB	0.515	0.949	0.143	0.024
Last 4 layers	1,130 MB	0.537	0.948	0.144	0.025
20 layers	3,965 MB	0.604	0.956	0.085	0.016
Full	5,735 MB	0.638	0.962	0.065	0.021

probabilities must be recalibrated before a threshold can use them.

9. Discussion

What a fleet should learn. Across the four options, the largest gain per byte came from the simplest step: fusing the tools' answers and correcting the threshold for the deployment's defect share (177.0 to 107.1), which needs no reinforcement learning and only a mean over posteriors from the fleet. A routing controller that sees the classifier's full output gains 40 risk points over a static cascade for about 7 MB when labels are available, and remains above the cascade with a 5% audit. A tool-selection policy on top of the fusion saves time but not risk. Federating perception improves defect ranking for gigabytes and needs recalibration. Using a VLM as the agent that reads tool outputs made reports worse in every configuration we tried.

Why the learned policy ties with the cascade. The fused probability already summarizes what the tools agree on, and its distance to the threshold is the main signal for calling another tool; a cascade uses exactly this signal. The oracle gap (105 against 65 at 3 s) comes from items where one particular tool is right and the others are wrong, and nothing in the state predicts which. Features such as tool disagreement on similar items, per-component tool reliability or image resolution relative to the component are

the next candidates. The learned policy also has practical merits a cascade lacks: it adapts to the budget and the link without retuning, and FedAvg trains it with lower variance than any single drone.

Limitations. Tool answers are cached and the fleet, the network and the onboard times are simulated; the Jetson factor of 9 is an estimate, and the risk weights are assumptions. The tool-selection and drone experiments use 3 seeds, the agentic and perception experiments one seed. The tool-selection report covers the defect decision only, not the component and condition questions. FedAvg for tool selection is computed in process; it equals FedAvg in Flower, which we used for the agentic experiments. The fusion is fitted with labels from validation towers, which a deployed fleet would need to collect. All data come from one benchmark and one asset type.

10. Conclusion

We asked what an inspection drone fleet should learn to use its perception tools well. A VLM that reads its tools' outputs got worse with tools, with and without PPO and FedAvg. Fusing the tools' answers with a label-free threshold lowered the risk from 177.0 to 107.1 per 1,000 items. A PPO policy that chooses which tool to call matched calling every tool in 30% less time, and FedAvg trained it as well as central training with 28 KB per update, but it did not beat a one-line uncertainty cascade, also when drones differed in components and network. The gap to an oracle that knows the right tools per item remains large, and closing it requires better per-item evidence rather than a better learner.

Data and code availability

The code, the cached tool answers and the result files are in the project repository, branches vision-fl and agentic-ppo-v2 (to be made public on acceptance). ElecVQA-Bench and InsPLAD are public [19, 20]. The laya-vision weights are published under CC BY-NC-SA 4.0 [6].

Declaration of competing interest

The author declares no competing interests.

CRediT authorship contribution statement

Mahdi Khashan: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing.

Declaration of generative AI use

During the preparation of this work the author used an AI assistant to write code, analyse results and draft text. The author reviewed and edited the content and takes full responsibility for the publication.

Acknowledgments

Experiments in this work were run and tracked with OpenResearch [24].

References

[1] R. Farahani, N. Hossein Motlagh, Z. Azimi, C. Timmerer, L. Carnevale, S. Tarkoma, S. Dustdar, From pixels to semantics: Edge AI for UAV-based critical infrastructure inspection, arXiv:2609.18448 (2026).

[2] R. Farahani, Z. Azimi, I. Murturi, A. Goknil, S. Sen, C. Timmerer, S. Dustdar, EVLM: Intent-driven edge vision language model for UAV-based power line inspection, in: 2026 IEEE International Conference on Edge Computing and Communications (EDGE), 2026. https://doi.org/10.1109/EDGE72783.2026.00014.

[3] Z. Azimi, R. Farahani, S. Dustdar, C. Timmerer, Cloud, edge, or split? Profiling onboard and split vision-language model deployment for drone AI, arXiv:2609.25415 (2026).

[4] A. Marafioti, et al., SmolVLM: Redefining small and efficient multimodal models, arXiv:2504.05299 (2025).

[5] Convai Innovations, Laya, software (2026). https://github.com/NandhaKishorM/laya.

[6] Laya Vision, software, commit 9e1e241 (2026). https://github.com/r33drichards/laya-vision; weights https://huggingface.co/thaitea/laya-vision.

[7] Sabanovic, Maliakel, Brandic, INAR-VL: Input-aware routing for edge-cloud vision-language inference, arXiv:2605.18853 (2026).

[8] H. Zhang, S. Cha, H. B. Beytur, K. S. Chan, G. de Veciana, H. Vikalo, Online learning for multi-layer hierarchical inference under partial and policy-dependent feedback, arXiv:2603.04247 (2026).

[9] W. Wei, T. Yang, H. Chen, Y. Zhao, F. Dernoncourt, R. A. Rossi, H. Eldardiry, Learning to route LLMs from bandit feedback: One policy, many trade-offs, arXiv:2510.07429 (2025).

[10] A. Agarwal, H. B. McMahan, Z. Xu, An empirical evaluation of federated contextual bandit algorithms, arXiv:2303.10218 (2023).

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 (2017).

[12] M. Saerens, P. Latinne, C. Decaestecker, Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure, Neural Computation 14 (1) (2002) 21–41.

[13] D. J. Beutel, et al., Flower: A friendly federated learning research framework, arXiv:2007.14390 (2020).

[14] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.

[15] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, V. Chandra, Federated learning with non-IID data, arXiv:1806.00582 (2018).

[16] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, in: Proceedings of Machine Learning and Systems (MLSys), 2020.

[17] Askin, et al., Federate the router: Learning language model routers with sparse and decentralized evaluations, arXiv:2601.22318 (2026).

[18] Solat, Lee, Seif, Niyato, Poor, FedHLM, IEEE Internet of Things Journal (2025). https://doi.org/10.1109/jiot.2025.3618095.

[19] ElecVQA-Bench, dataset and code repository (2026). https://github.com/GhostDino/elecvqa-bench.

[20] A. L. B. Vieira e Silva, et al., InsPLAD: A dataset and benchmark for power line asset inspection in UAV images, International Journal of Remote Sensing (2023). arXiv:2311.01619.

[21] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. R. Alface, T. Bostoen, F. De Turck, HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks, IEEE Communications Letters 20 (11) (2016) 2177–2180.
[22] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
[23] E. J. Hu, et al., LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.
[24] alphaXiv, OpenResearch: The local-first harness and workspace for research agents (2026). https://github.com/alphaXiv/OpenResearch.